

SELF BALANCING ROBOT — PROJECT REPORT

Team Members

Aks Srivastava – 2024CHB1056

Iia Bansal – 2024CHB1069

Ritika – 2024CHB1077

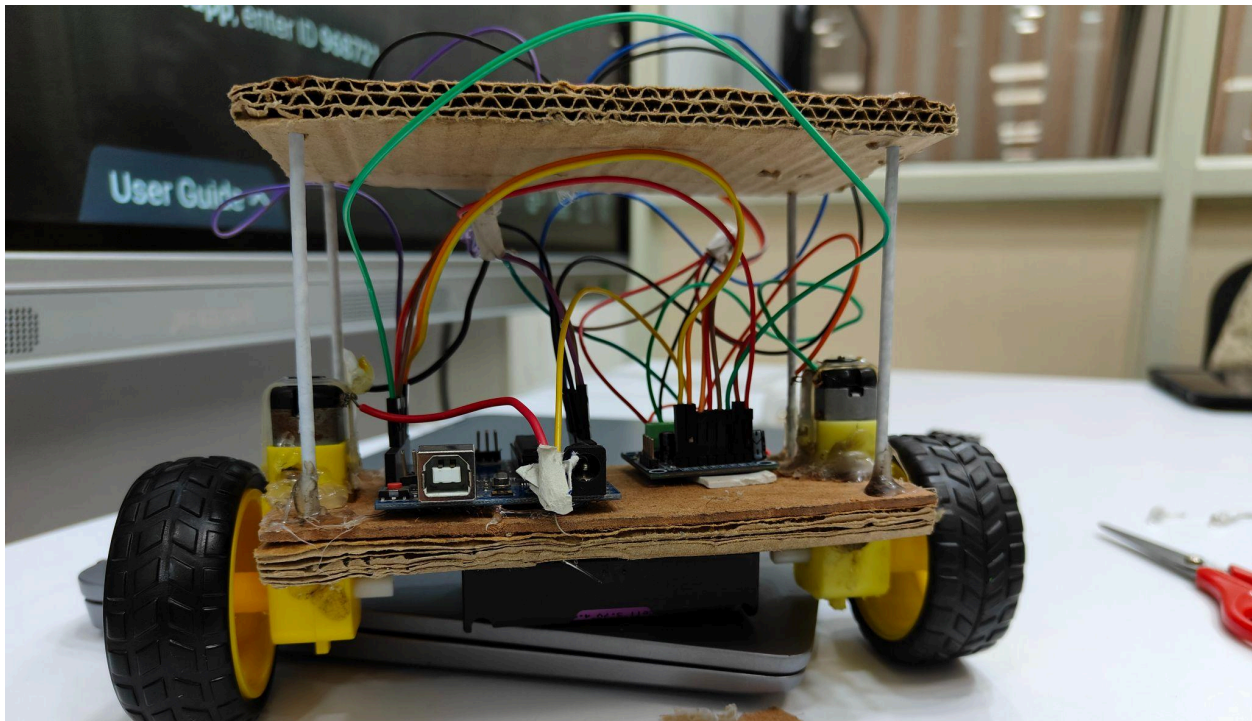
Roushani Kumari – 2024CHB1079

Tannu Agarwal – 2024CHB1088

1. Overview

Our self-balancing robot is a two-wheeled, differential-drive robotic platform designed to remain upright while in motion or stationary. The system continuously receives angular tilt data from the MPU6050 sensor and processes it using a PID control loop.

The PID controller balances the robot by adjusting motor speeds based on tilt error. Proportional control addresses large deviations, derivative control minimizes oscillations, and integral control compensates for long-term drift, ensuring stability over time. The placement of components and weight distribution allow the robot to achieve active stabilization using real-time feedback.



2. Objectives

- To build a two-wheeled self-balancing robot using a PID-based control system.
- To interface MPU6050 with Arduino for continuous tilt measurement.
- To understand and implement real-time feedback control.
- To design a compact and efficient prototype for balancing applications.
- To integrate hardware and software components into a functional robotic platform.

3. Problem Statement

A two-wheeled robot is naturally unstable and requires continuous correction to maintain balance. The objective of this project is to develop a system capable of stabilizing itself using feedback from a motion sensor and a control algorithm. The challenge lies in achieving the right balance of sensitivity, motor response and algorithmic tuning to handle disturbances and maintain equilibrium.

4. System Model

Hardware Components

- Arduino UNO – Main microcontroller running PID algorithm.
- MPU6050 IMU – Measures tilt angle and angular velocity and sends data for balance correction.
- L293D Motor Driver – Controls BO motors through PWM signals generated by Arduino.
- BO (battery operated) Motors (2) – Provide driving torque needed to maintain balance.
- Wheels (2) – To drive the bot
- Battery, Holder & Switch – Power supply system.
- Connecting Wires – Circuit connection & assembly.

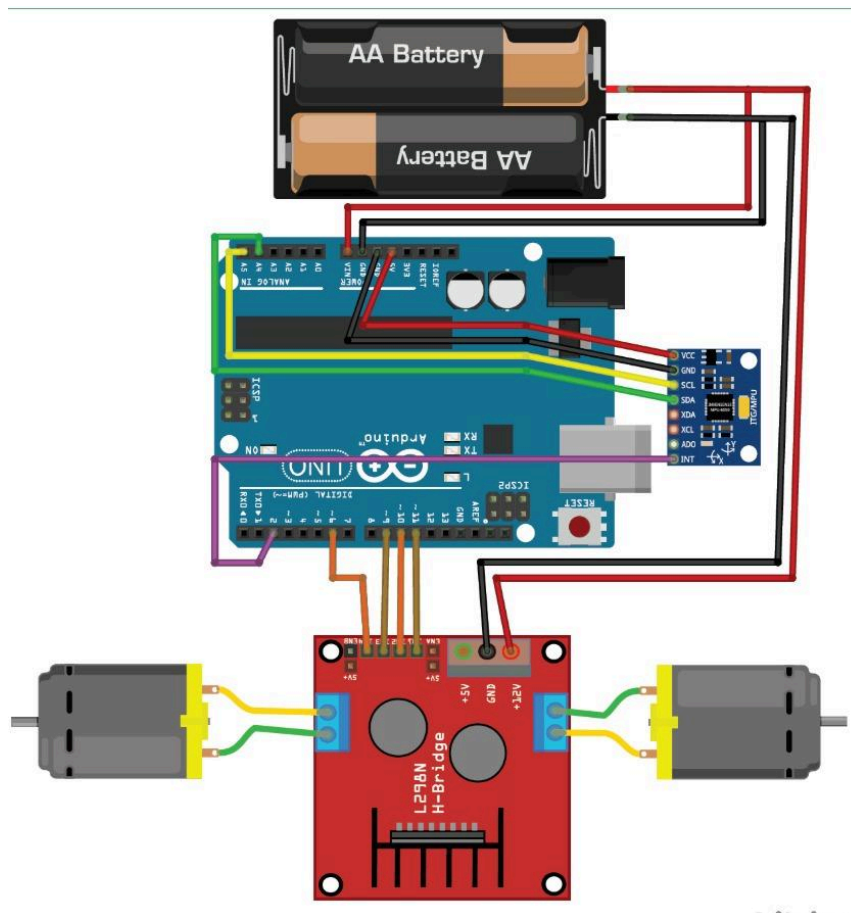
Software Components

- Arduino IDE – used for coding, uploading & debugging.

5. Working Principle

- The functioning of the robot is based on feedback control theory, where continuous monitoring and correction allow it to stay balanced.
- The MPU6050 sensor constantly measures the robot's tilt and provides accelerometer and gyroscope readings.

- The Arduino reads this sensor data and calculates the error, which is defined as the difference between the ideal upright angle and the robot's current angle.
- The PID controller processes this error and determines how much corrective power should be sent to the motors to reduce the imbalance.
- The motor driver then adjusts the direction and speed of the motors according to the PID output, ensuring that the robot attempts to stand upright again.
- The wheels move in response to the control signals, shifting the robot in the direction required to regain balance.
- This sensing-and-correcting loop occurs several hundred times every second, enabling fast and precise stabilization.
- The Proportional term helps correct large deviations immediately, the Integral term counteracts slow and gradual drift, and the Derivative term smoothens the response by reducing overshoot and wobbling.
- Combined together, these control actions allow the robot to balance itself effectively and remain upright even when small disturbances are applied.



6. Challenges & Problems Faced

Throughout development, several issues tested our troubleshooting ability:

- MPU6050 angle values dropped unexpectedly during initial tests (filtered angle, error, and power fluctuating).
- VCC and GND wires short-circuited due to accidental contact, causing the **battery holder and motor driver to short**.
- We later discovered the motor driver was already shorted, which caused **the newly attached battery holder to short again**, leading to component replacement.
- Lab was closed during this period, delaying component replacement and progress.
- Motor spinning speeds were inconsistent, causing the bot to trace a circular path instead of balancing.
- Arduino UNO pin **D6 failed**, so we had to swap board during final debugging.
- PWM output malfunction led us to replace motor driver completely.
- Due to difference in the speed of the two motors, the PID tuning wasn't working efficiently.
- As the motors were old, while tightening a wire, the loop connecting the wire and motor broke.

7. Future Improvements

We plan to increase capabilities in the next version through:

1. Bluetooth / Wi-Fi control

Remote handling allows tele-operation of robot, speed control, direction control and data visualization.

2. Li-ion rechargeable battery pack

Integration of Li-ion rechargeable battery packs with onboard charging module.

3. Acrylic/Metal body

Use of metal or acrylic body for improved durability & vibration reduction.

4. Ultrasonic obstacle detection

Adding ultrasonic obstacle detection for autonomous movement.

8. Conclusion

This project successfully demonstrates a working self-balancing robot driven by IMU feedback and PID control. Despite hardware failures and debugging challenges, the final system achieved upright balance and stable response. The project helped us understand real-time control mechanisms, sensor processing and robotic actuation. With future upgrades like wireless control and improved body material, the robot can evolve into a more robust and feature-rich version.